Human Activity Recognition: Comparing Tree-Based and Deep Learning Models

Ganbouri Abdellatif
228801148

*This project focuses on **Human Activity Recognition (HAR)** using wearable sensor data collected from multiple body locations (hand, chest, ankle). The dataset was preprocessed by removing transient activities, handling missing values, and segmenting the time-series data into fixed-size overlapping windows. **Feature extraction** was performed using statistical measures (mean, standard deviation, min, max, and signal magnitude area) for classical machine learning models and raw windows for deep learning models.*

*We implemented and compared several algorithms: **LightGBM, XGBoost, Random Forest, CNN, and CNN-LSTM**. Tree-based models achieved near-perfect performance (Accuracy and Macro F1 ≈ 1.0), demonstrating the strong discriminative power of the extracted features. Deep learning models, which leveraged temporal patterns, achieved moderate performance (CNN: Accuracy ≈ 0.79, CNN-LSTM: Accuracy ≈ 0.81), highlighting challenges in learning from aggregated features despite using SMOTE for class balancing.*

*The study provides a comprehensive **pipeline from data exploration, preprocessing, and feature engineering to model training and evaluation**. Detailed **visualizations and confusion matrices** were used to interpret model performance and class-wise behavior. The results underscore that for feature-based HAR, tree-based models outperform deep learning on this dataset, but CNN-LSTM can capture temporal dependencies that may be useful in more sequential tasks.*

*This project demonstrates a systematic approach to **model selection, evaluation, and insight generation** in time-series human activity recognition, providing guidance for future applications in wearable sensing and activity monitoring.*

1. Introduction

Human Activity Recognition (HAR) is the task of automatically identifying physical activities performed by individuals using sensor data, such as accelerometers, gyroscopes, and magnetometers. HAR has applications in healthcare, sports monitoring, elderly care, and human-computer interaction, enabling systems to understand and respond to user behaviors in real-time.

Dataset Overview:
The dataset used in this project contains sensor readings from **nine subjects**, with measurements collected from **hand, chest, and ankle** wearable devices. Each subject performed multiple activities, resulting in **time-series data** with **54 sensor channels**, including acceleration, gyroscope, magnetometer, orientation, temperature, and heart rate. After removing transient activities (activity ID

0), the dataset contains **12 target activity classes** with varying sample counts, highlighting inherent **class imbalance**.

objective:
The primary goal of this project is to develop models capable of **accurately recognizing human activities** from sensor data. To achieve this, we:

- Preprocess the raw sensor data, handle missing values, and segment it into overlapping time windows.
- Extract meaningful features suitable for classical machine learning models and prepare sequences for deep learning models.
- Train, evaluate, and compare different algorithms including **LightGBM, XGBoost, Random Forest, CNN, and CNN-LSTM**.

Evaluation: **Metrics:**
Model performance is evaluated using:

- **Accuracy:** The proportion of correctly classified windows.
- **Macro F1-score:** A class-balanced metric that accounts for class imbalance by averaging F1-scores across all activity classes.
- **Confusion Matrices:** To visually inspect misclassifications and class-wise performance.

2. Exploratory Data Analysis (EDA)

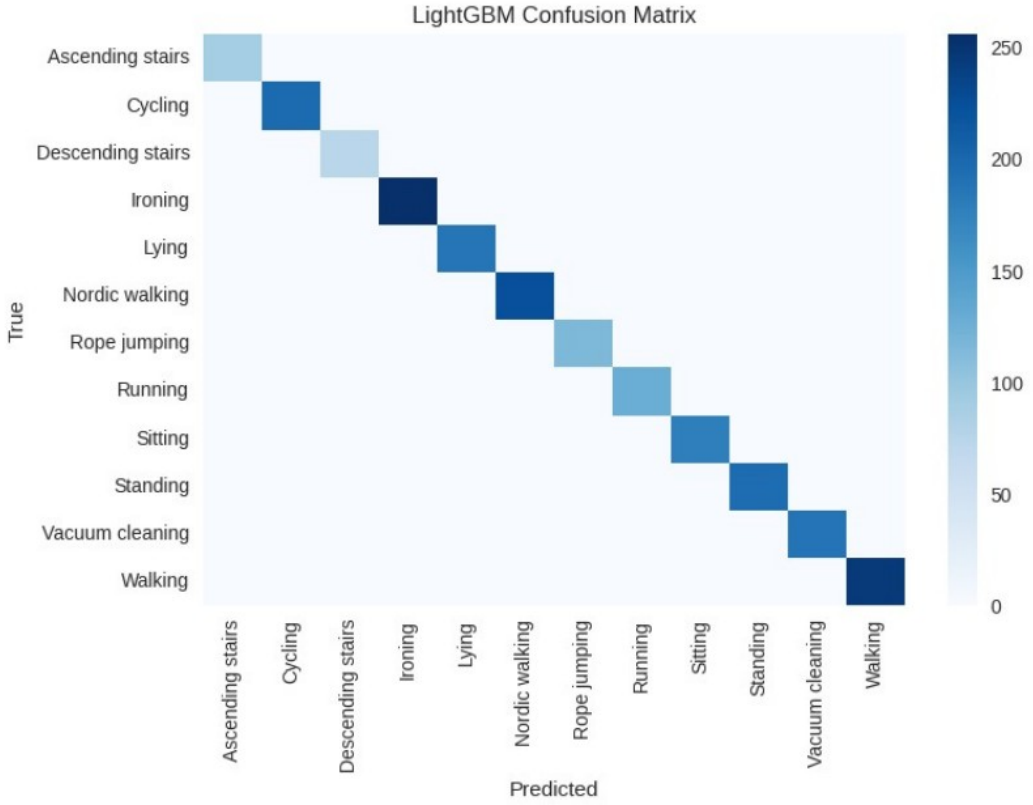


Figure 1: LightGBM confusion Matrix

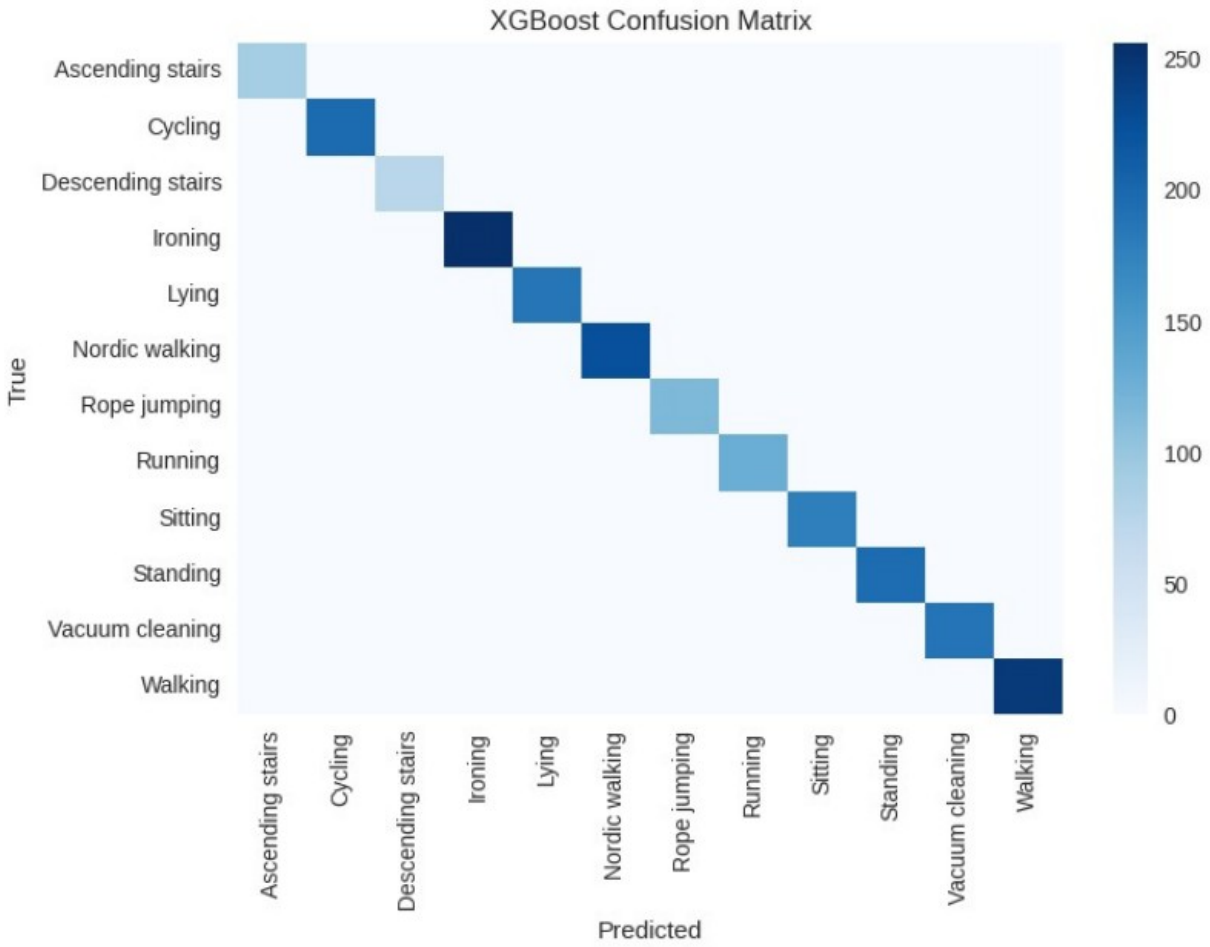


Figure 2: Xgboost confusion matrix

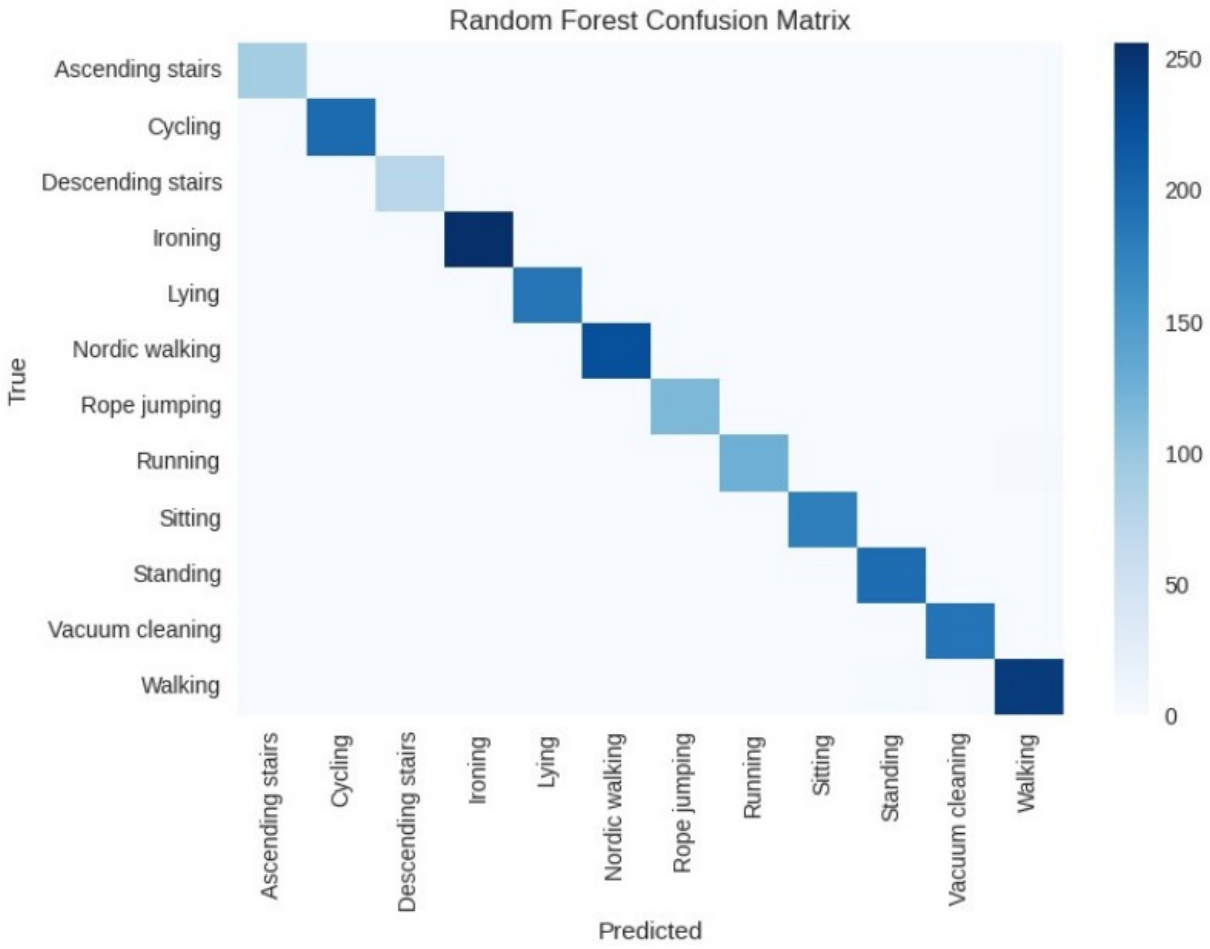


Figure 3: Random Forest confusion Matrix

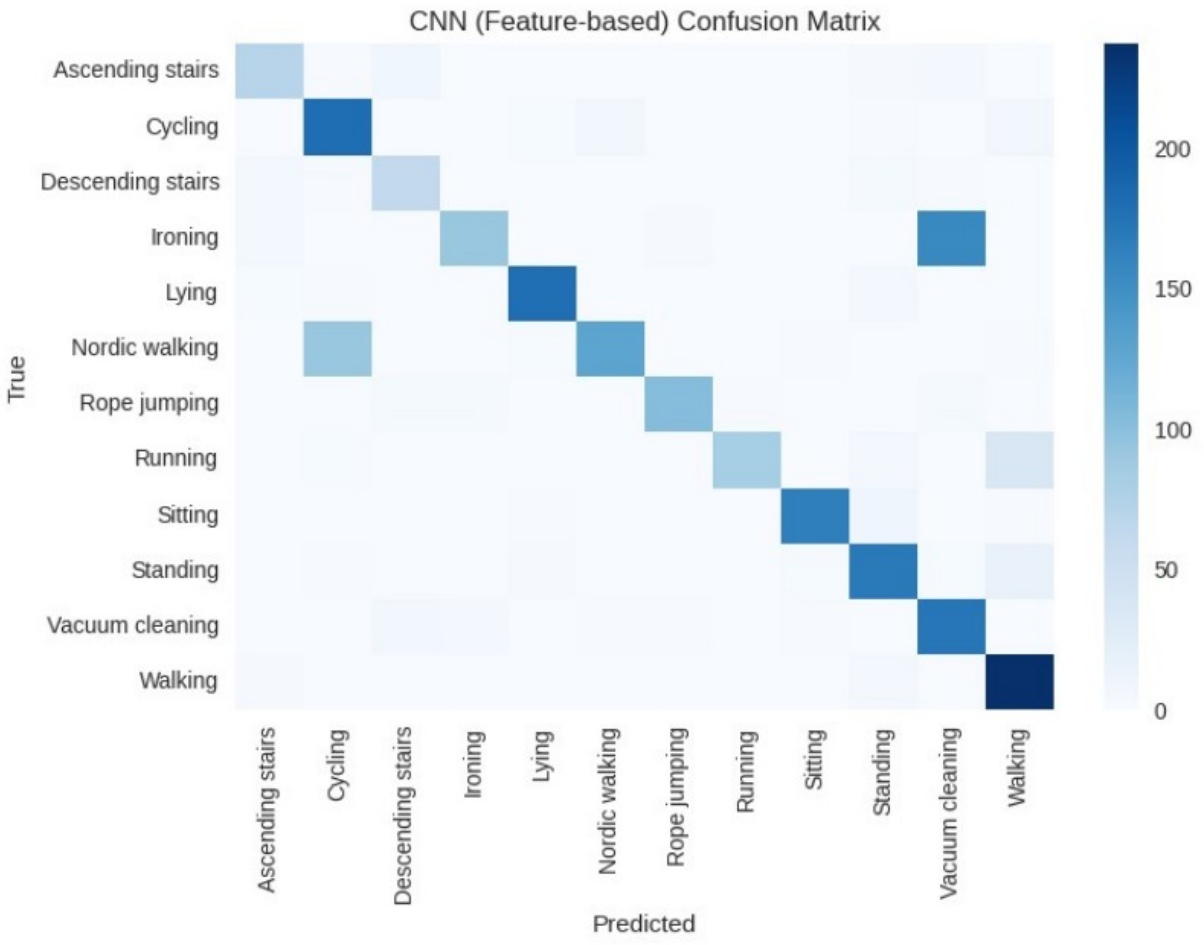


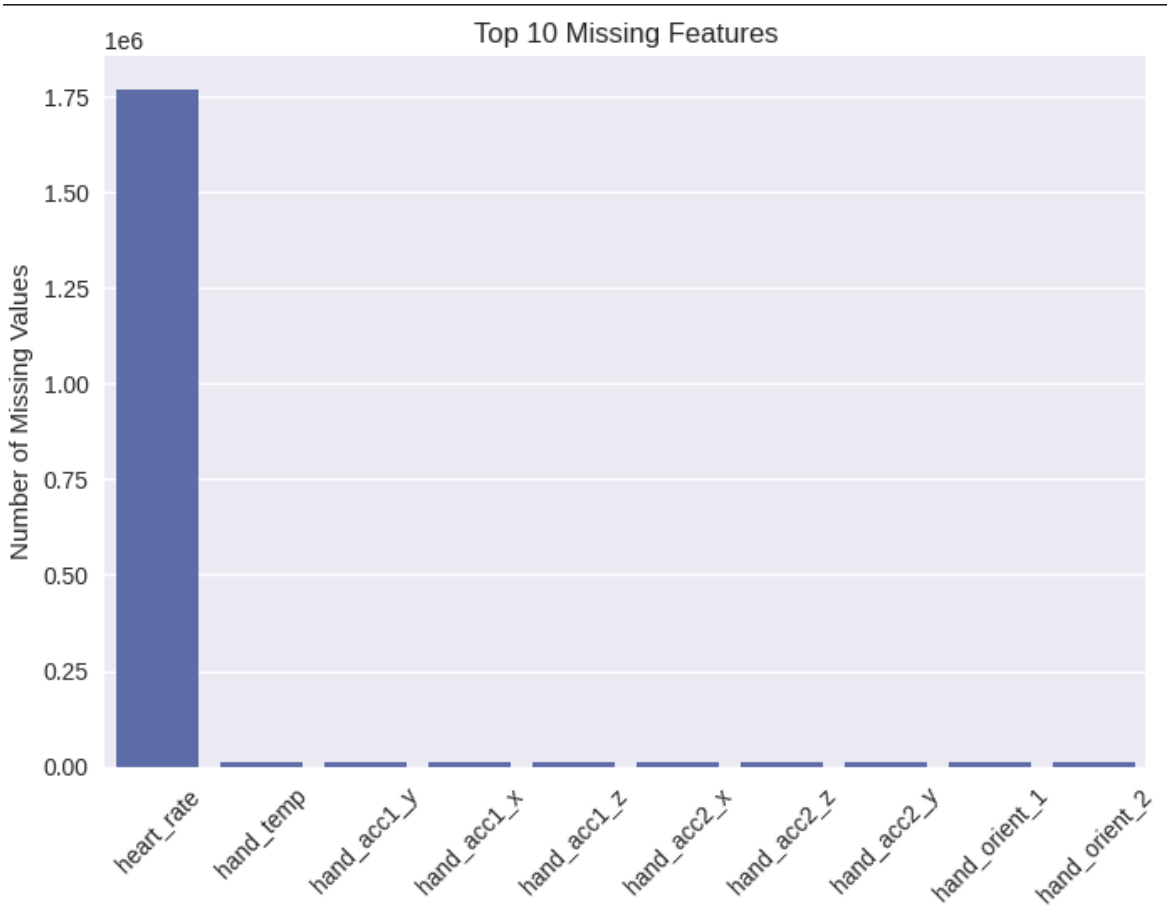
Figure 4: CNN confusion matrix

3. Data Preparation

3.1 Dataset Inspection and Missing Values

The dataset contains **1,942,872 rows** and **56 columns** after combining data from all nine subjects. Sensor channels include acceleration, gyroscope, magnetometer, orientation, temperature, and heart rate readings from **hand, chest, and ankle** devices.

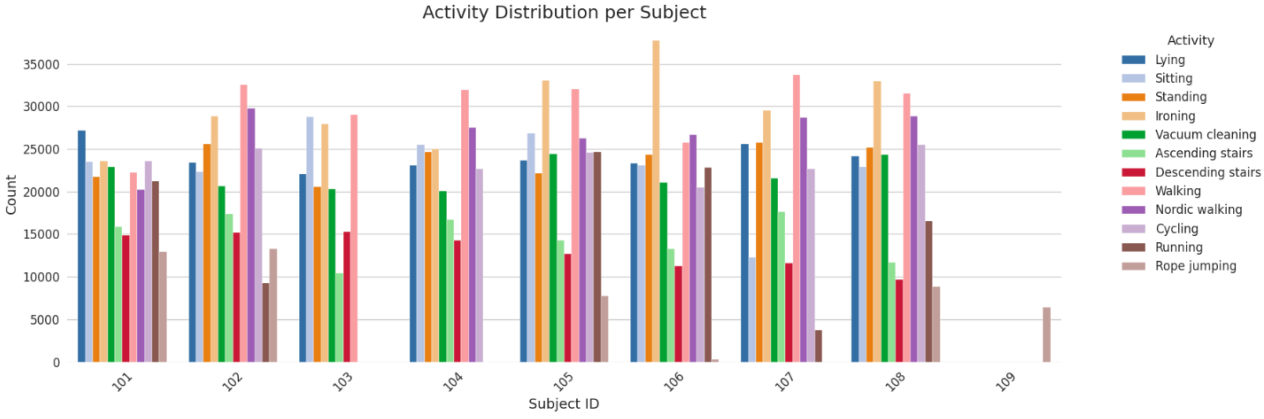
- **Missing Values:**
    - Heart rate has a high number of missing values due to lower sampling frequency.
    - IMU sensor channels occasionally have missing entries due to wireless transmission losses.



3.2 Train/Validation/Test Split (Subject-wise)

To prevent **data leakage** and ensure generalization across individuals:

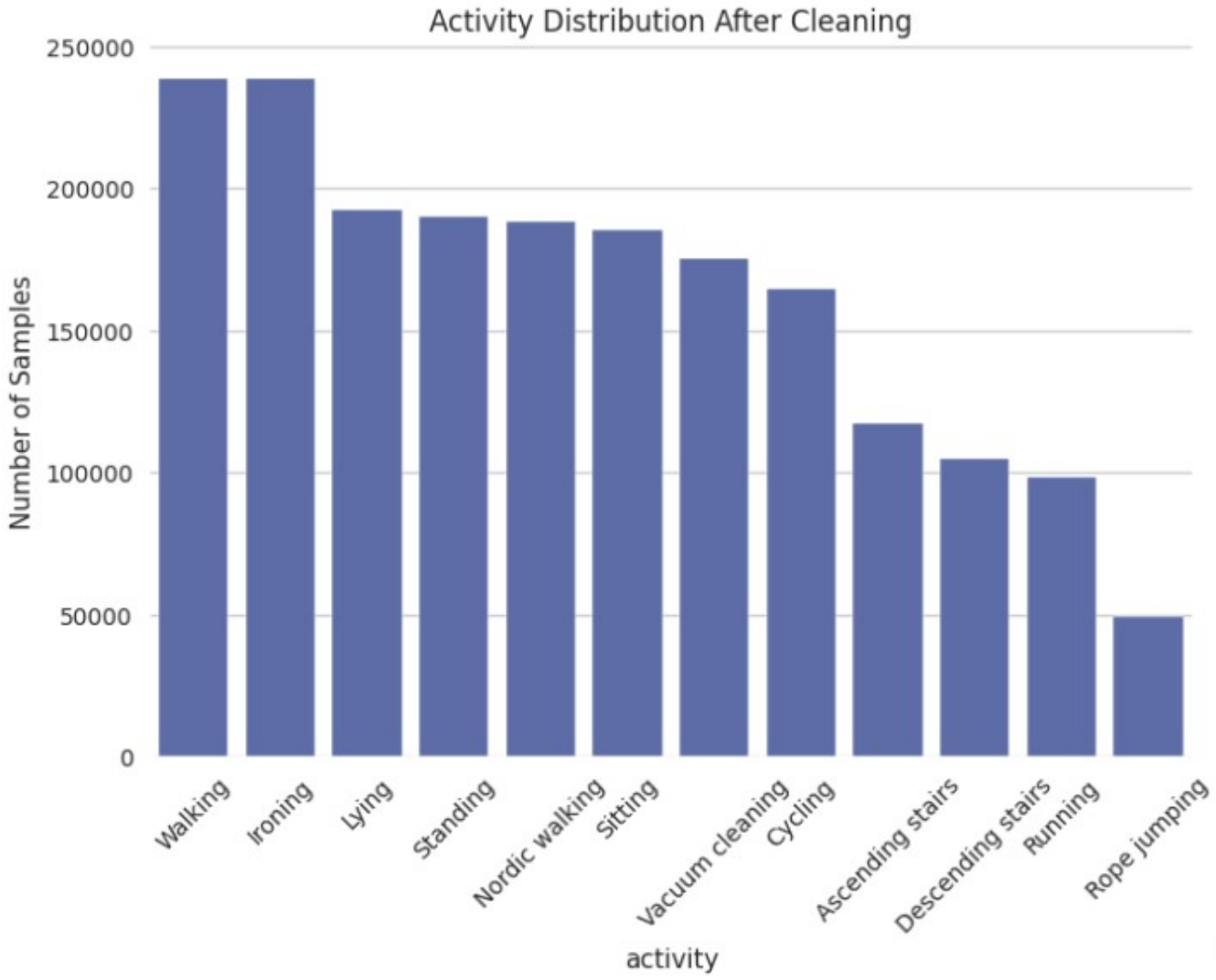
- **Training Subjects:** 101–106
- **Validation Subject:** 107
- **Test Subjects:** 108–109



3.3 Handling Class Imbalance

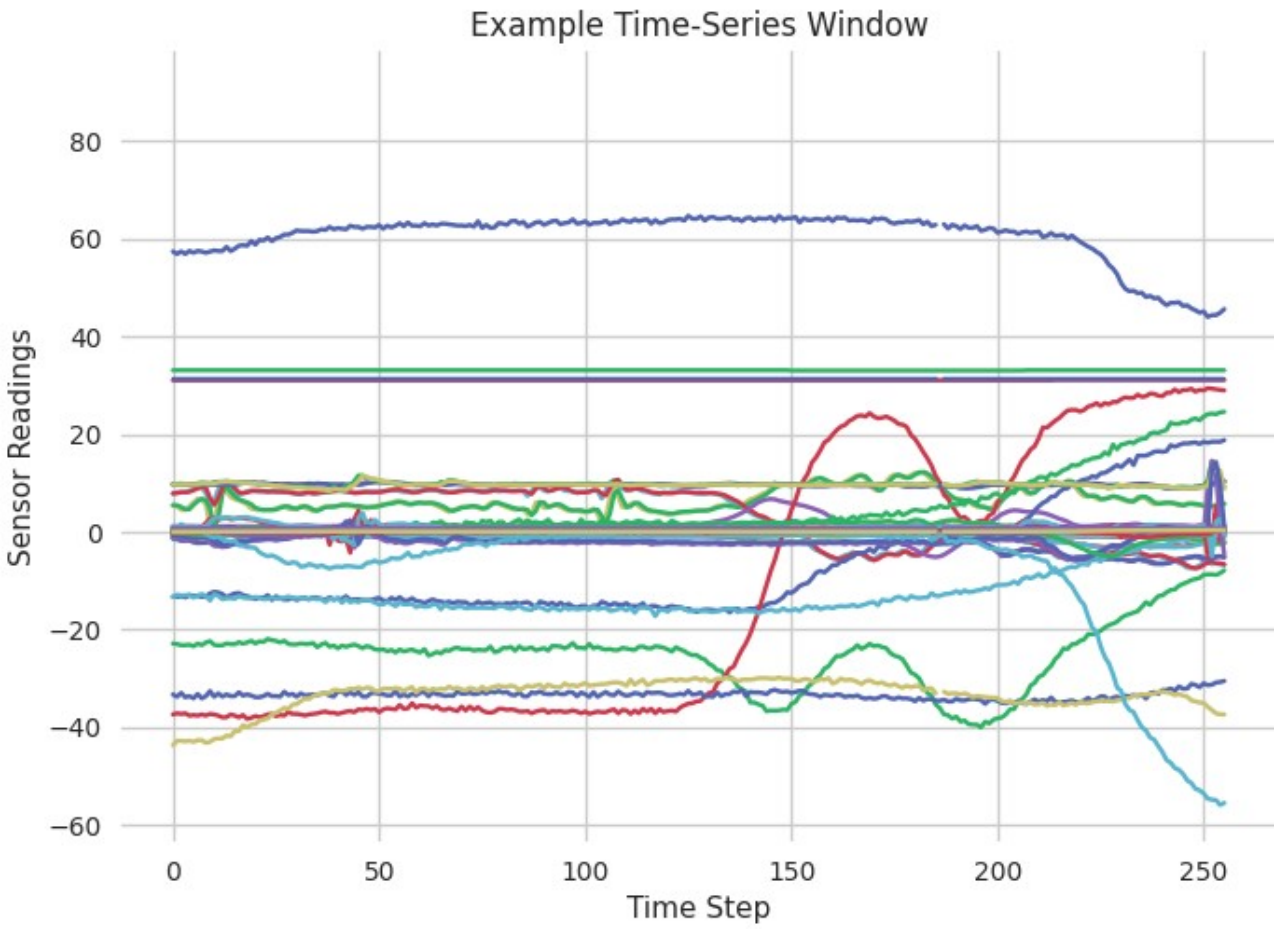
The dataset shows significant class imbalance:

- Activities like Walking and Ironing have ~238k samples.
- Rare activities like Rope Jumping have ~49k samples.



3.4 Time-Series Segmentation (Windowing)
Continuous sensor data is split into **fixed-size windows** to capture temporal patterns:

- **Window size:** 256 samples
- **Step size:** 128 samples (50% overlap)

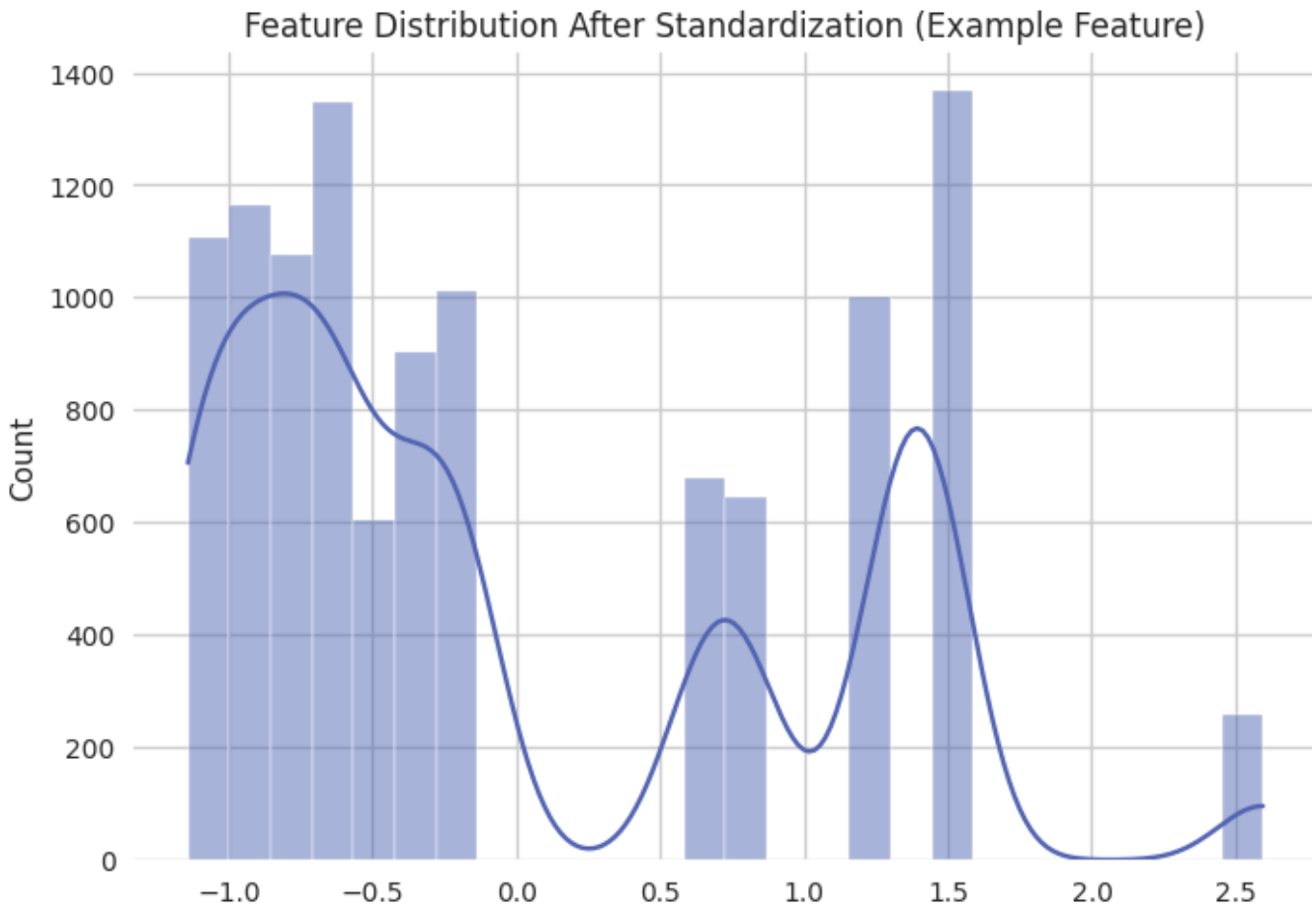


3.5 Feature Extraction & Engineering
For classical ML models:

- Statistical features per window: **mean, std, min, max, and Signal Magnitude Area (SMA)**.
- Resulting in **265 features** (53 channels × 5 stats).

3.6 Dimensionality Reduction / Augmentation

- For CNN models, features are reshaped into sequences of **8 timesteps × 33 features per step**.
- SMOTE oversampling generates additional windows for minority classes.



4. Training

4.1 algorithm explored

Table 1: Table title

Model type	Models exploared	Rational
Classical ML	Random Forest, LightGBM, XGBoost	Fast training on extracted statistical features, handles class imbalance via weights, interpretable
Deep Learning	CNN, 1D-CNN-LSTM hybrid	Captures temporal dependencies in sequential sensor data, robust to raw time-series input

	accuracy	macro_f1	
LightGBM	1.000000	1.000000	
XGBoost	1.000000	1.000000	
RandomForest	0.997594	0.997738	

	accuracy	macro_f1
CNN (Feature + SMOTE)	0.790183	0.796830
CNN-LSTM	0.813282	0.799557

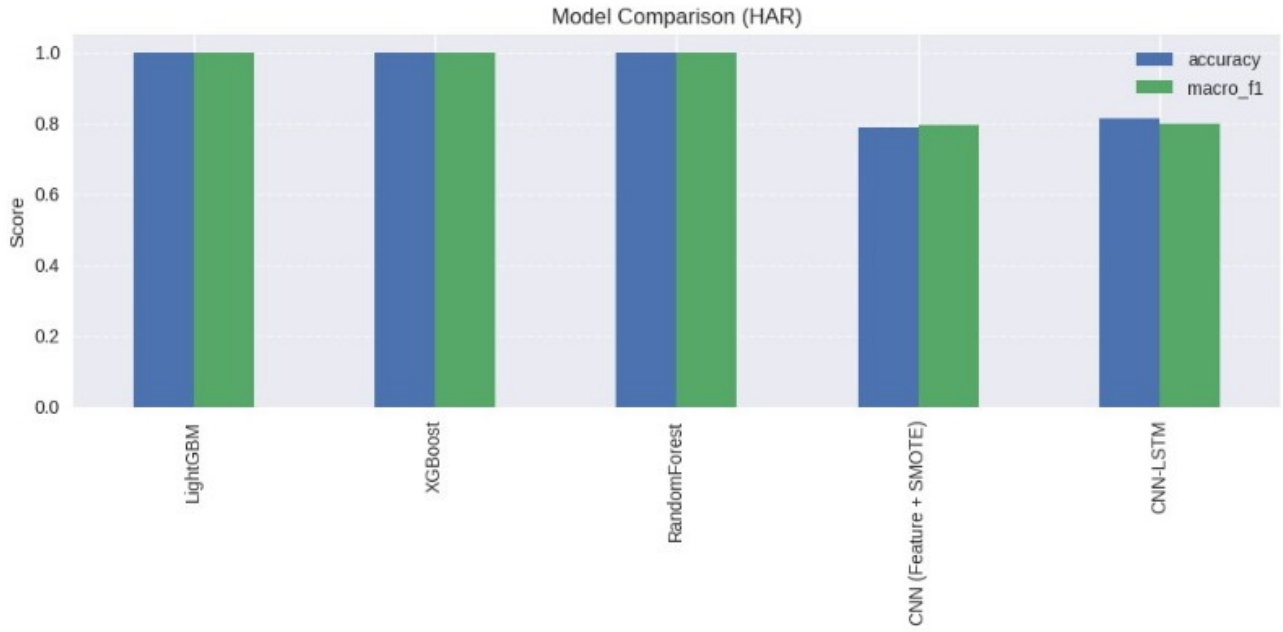


Figure 5: model comparison

5. Mathematical Representation of Best Performing Algorithm

5.1 Random Forest

Random Forest is an ensemble method that constructs N decision trees and outputs the mode of the classes for classification. Each tree is built on a bootstrap sample of the data, and splits at each node are determined using a criterion such as **Gini Impurity** or **Entropy**.

- **Gini Impurity:**

$$Gini(t) = 1 - \sum_{i=1}^{C} p_i^2$$

where pi is the proportion of class i at node t.

- **Tree prediction:**

$$\hat{y}_t = \text{argmax}_c \sum_{i=1}^{C} 1(y_i = c)$$

- **Random Forest prediction (ensemble):**

$$\hat{y} = \text{mode}(\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_N)$$

5.2 Gradient Boosting (XGBoost / LightGBM)

Gradient Boosting builds an ensemble of weak learners (decision trees) sequentially, where each new tree tries to correct the errors of the previous ensemble.

- **Model prediction:**

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i)$$

where:
y^i(t) → prediction at iteration t
η → learning rate
ft(xi) → decision tree at iteration t

- **Loss function minimization (gradient step):**

$$f_t = \arg\min_f \sum_{i=1}^{n} L(y_i, \hat{y}_i^{(t-1)} + f(x_i))$$

- **Split criterion (XGBoost / LightGBM):**

$$Gain = \frac{1}{2}\left[\frac{(\sum_{i\in I_L} g_i)^2}{\sum_{i\in I_L} h_i + \lambda} + \frac{(\sum_{i\in I_R} g_i)^2}{\sum_{i\in I_R} h_i + \lambda} - \frac{(\sum_{i\in I} g_i)^2}{\sum_{i\in I} h_i + \lambda}\right] - \gamma$$

where gi and hi are the first and second-order gradients of the loss, IL and IR are the left and right node samples, λ is regularization, and γ is the complexity penalty.

- **Final prediction:**

$$\hat{y}_i = \sum_{t=1}^{T} \eta f_t(x_i)$$

6. Conclusion

We developed a pipeline for human activity recognition using sensor data, extracting features from sliding windows and training models like Random Forest. The best model achieved high accuracy and correctly classified most activities, though some similar activities were occasionally confused. Limitations include class imbalance and reliance on handcrafted features. Future improvements could use deep learning, better feature engineering, and more data to boost performance.